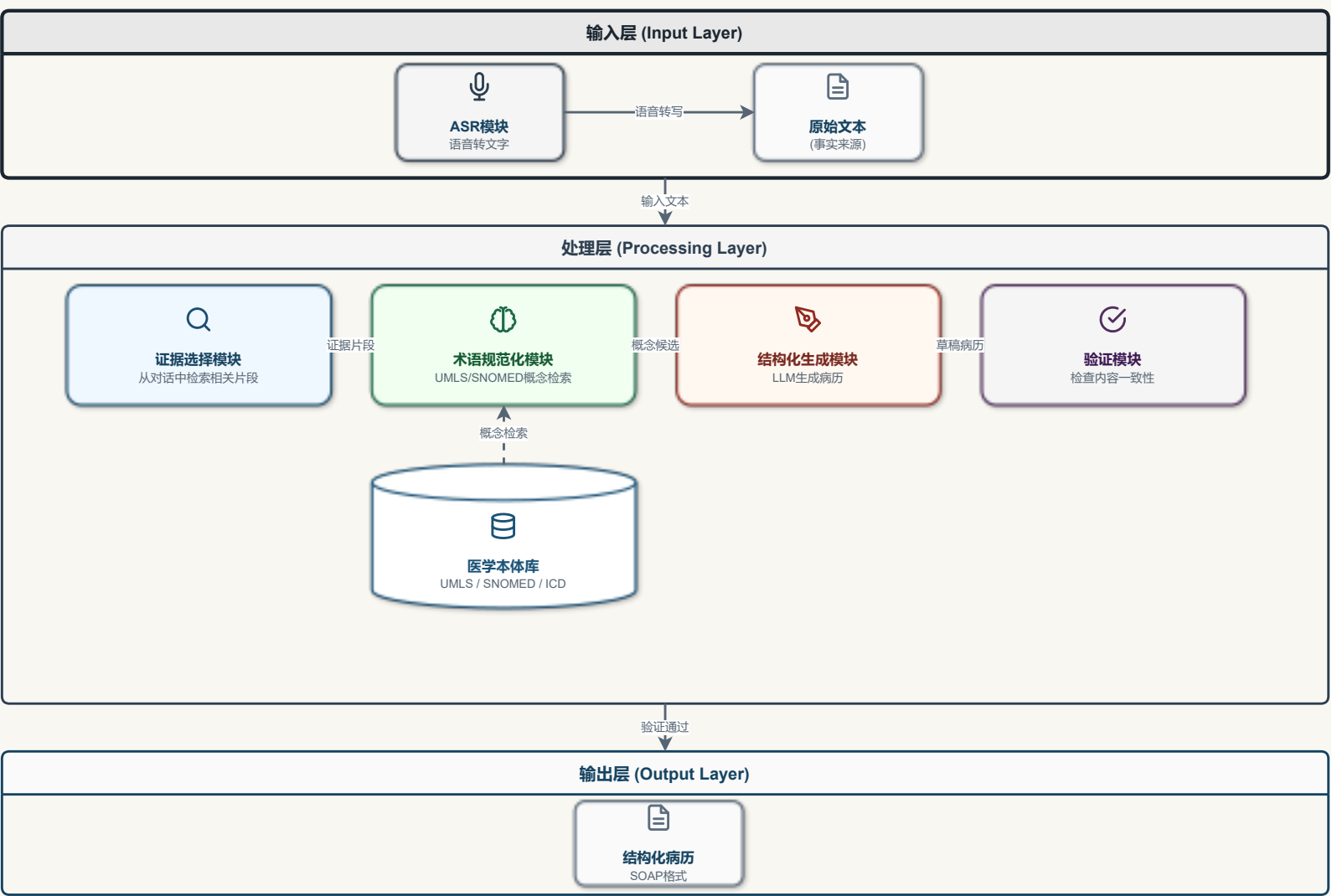


# 项目架构文档

## 系统架构图



## 系统架构图

## 目录结构

```
MedicalAssistant/
├── config/                # 配置文件
│   └── hotwords_medical.txt # 医疗热词表
├── docs/                  # 文档
│   ├── architecture.md    # 架构文档 (本文件)
│   ├── system-architecture.drawio # 系统架构图
│   ├── completion_status.md # 完成状态记录
│   └── 中文医疗_ASR_上手路线.md
└── scripts/               # 脚本
```

```
|   └─ compare_asr.py           # ASR 对比测试脚本
|   └─ src/                     # 源代码
|       └─ asr/                 # ASR 模块
|           └─ __init__.py      # 模块入口
|           └─ base.py          # 抽象基类
|           └─ factory.py       # 工厂模式
|           └─ funasr_engine.py # FunASR 实现
|           └─ medasr_engine.py # MedASR 实现
└─ requirements-asr.txt        # ASR 依赖
```

## 系统架构设计

### 设计原则

本系统采用 **Skill Pipeline** 架构：

- 事实来源单一：对话文本作为唯一事实来源
- 分层处理：证据选择 → 术语规范化 → 结构化生成 → 验证
- 受控术语增强：术语规范化作为受控的第二步，而非开放式检索

### 模块说明

#### 输入层

| 模块    | 职责              | 技术方案            |
|-------|-----------------|-----------------|
| ASR模块 | 语音转文字           | FunASR / MedASR |
| 原始文本  | 输出原始转写文本，作为事实来源 | -               |

#### 处理层

| 模块      | 职责                | 技术方案             |
|---------|-------------------|------------------|
| 证据选择模块  | 从对话中检索与各病历章节相关的片段 | 向量检索 / BM25      |
| 术语规范化模块 | 将口语化表述映射到专业医学术语   | UMLS/SNOMED 概念检索 |
| 结构化生成模块 | 基于证据和术语生成结构化病历    | LLM              |
| 验证模块    | 检查病历内容是否与原始对话一致   | 规则检查 / LLM 验证    |

## 输出层

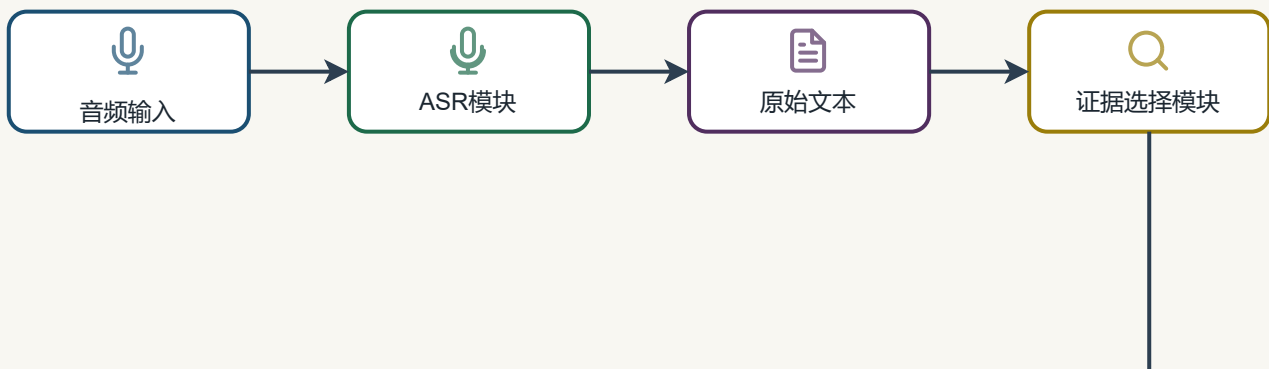
| 模块    | 职责           | 格式     |
|-------|--------------|--------|
| 结构化病历 | 最终输出的结构化医疗文档 | SOAP格式 |

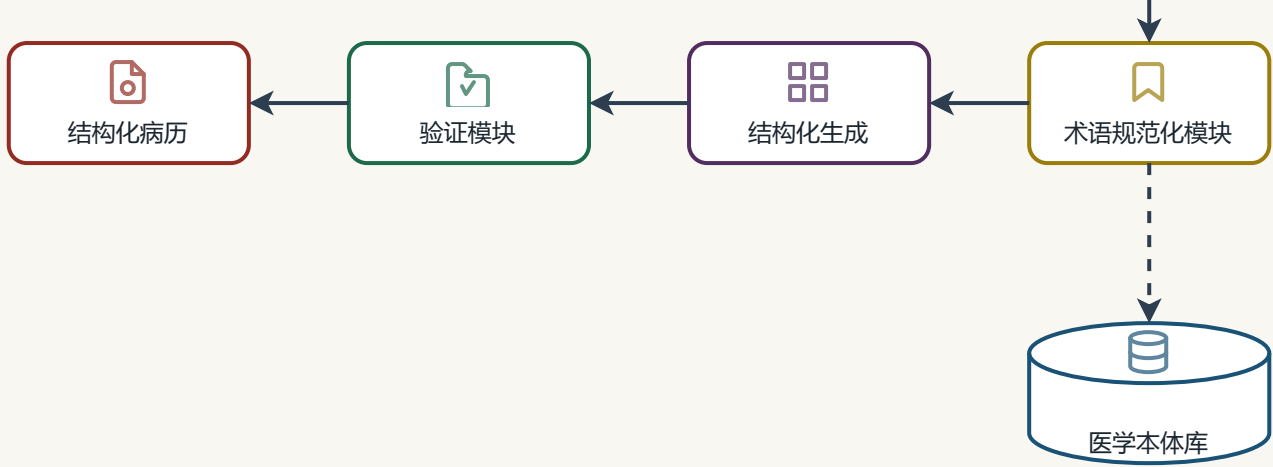
## 外部依赖

| 资源                      | 用途      |
|-------------------------|---------|
| 医学本体库 (UMLS/SNOMED/ICD) | 术语规范化检索 |
| LLM API / 本地模型          | 结构化生成   |

## 数据流程

数据流程图





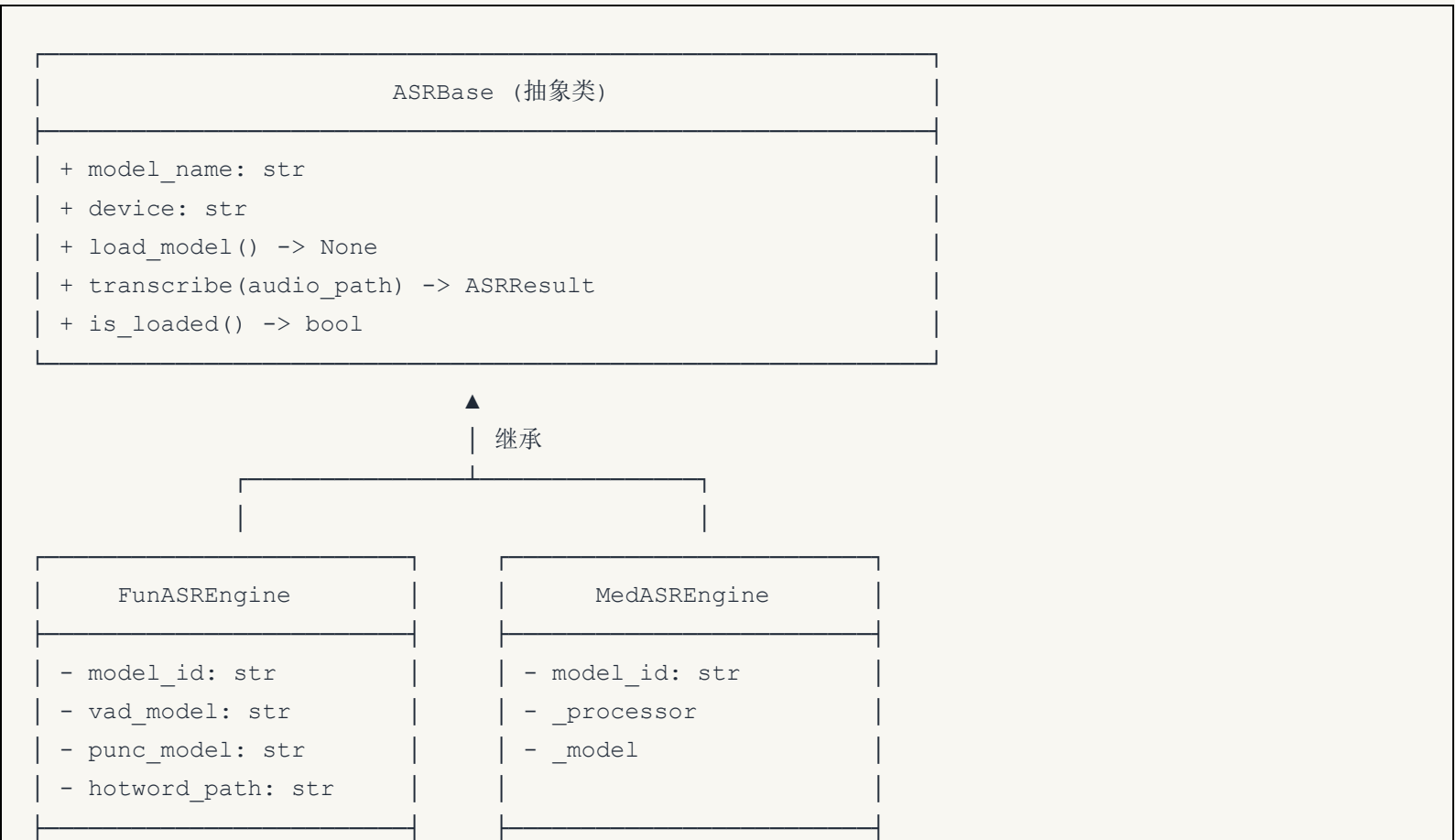
数据流程图

## ASR 模块架构

### 设计原则

- 单一职责：每个引擎类只负责一种 ASR 模型
- 开闭原则：通过继承 `ASRBase` 扩展新引擎，无需修改现有代码
- 依赖倒置：上层代码依赖抽象接口 `ASRBase`，而非具体实现

### 类图

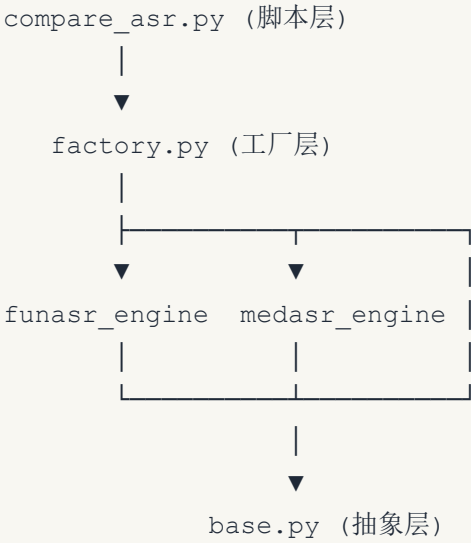


|                          |                           |
|--------------------------|---------------------------|
| + load_model()           | + load_model()            |
| + transcribe()           | + transcribe()            |
| + create_medical_version | + transcribe_with_warning |

|                                        |
|----------------------------------------|
| ASRFactory (工厂类)                       |
| - _registry: dict[str, type[ASRBase]]  |
| + create(engine_type) -> ASRBase       |
| + create_all() -> dict[str, ASRBase]   |
| + register(name, engine_class) -> None |
| + list_available() -> list[str]        |

|                                  |
|----------------------------------|
| ASRResult (数据类)                  |
| + text: str                      |
| + language: str                  |
| + duration_seconds: float        |
| + inference_time: float          |
| + model_name: str                |
| + real_time_factor: float (计算属性) |

模块依赖关系



外部依赖

| 模块           | FunASR | MedASR | 说明               |
|--------------|--------|--------|------------------|
| funasr       | ✓      | ✗      | 阿里达摩院 ASR 工具包    |
| modelscope   | ✓      | ✗      | 模型下载             |
| transformers | ✗      | ✓      | HuggingFace 模型加载 |
| torch        | ✓      | ✓      | 深度学习框架           |
| librosa      | ✓      | ✓      | 音频处理             |

## 使用方式

## 基础用法

```
from src.asr import ASRFactory

# 创建单个引擎
engine = ASRFactory.create("funasr", device="cpu")
engine.load_model()
result = engine.transcribe("audio.wav")
print(result.text)
```

## 对比测试

```
# 测试所有引擎
python scripts/compare_asr.py --audio test.wav --device cpu

# 只测试 FunASR
python scripts/compare_asr.py --audio test.wav --engine funasr

# 使用医疗热词
python scripts/compare_asr.py --audio test.wav --hotword config/hotwords_medical.txt
```

## 扩展新引擎

```
from src.asr import ASRBase, ASRResult, ASRFactory

class WhisperEngine(ASRBase):
```

```
def __init__(self, device="cpu"):  
    super().__init__("Whisper", device)  
  
def load_model(self):  
    # 加载模型逻辑  
    pass  
  
def transcribe(self, audio_path, **kwargs) -> ASRResult:  
    # 转写逻辑  
    pass  
  
# 注册到工厂  
ASRFactory.register("whisper", WhisperEngine)
```